

Arquitectura de Microservicios con Contenedores

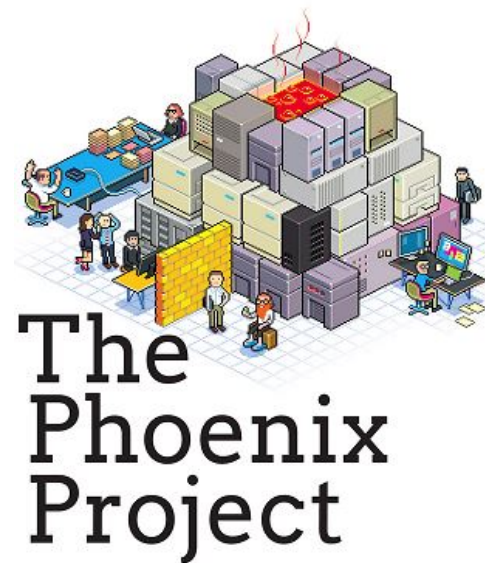
Docentes

- Ing. Federico Castañeda
- Ing. Carlos Maceira
- Ing. Franco Rupnik

DevOps

DevOps

- DevOps es una serie de prácticas, herramientas y filosofía de trabajo que busca automatizar todos los procesos entre el desarrollo y la puesta en producción de las aplicaciones.
- Dev proviene de Development y Ops de Operations. Muchas veces también se define el término DevSecOps, al incorporar Security como una de las prioridades de este rol.



The Phoenix Project

Beneficios de DevOps

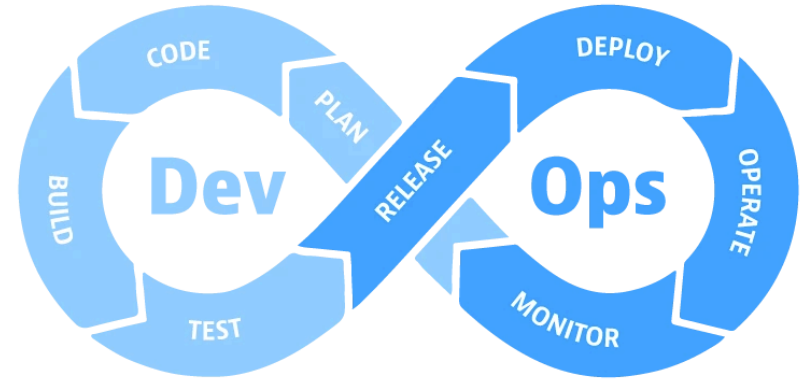
- Velocidad en los procesos de despliegue. Mayor frecuencia y menos errores en los procesos.
- Colaboración entre equipos. Las prácticas de DevOps se basan en procesos efectivos desarrollados por equipos interdisciplinarios.
- Desarrollos más rápidos. Al poder hacer releases más frecuentes, se pueden hacer desarrollos más chicos, de modo de implementar menor pero más seguido.
- Calidad y confianza en el proceso de despliegue permite la implementación de nuevas funcionalidades sin afectar el servicio en producción.
- Seguridad, incorporada desde el proceso de desarrollo y controlado a lo largo del proceso.

Conceptos de DevOps

Plan. Planificación de la siguiente iteración de cambios.

Involucra la planificación de nuevas funcionalidades, y/o correcciones de las métricas observadas.

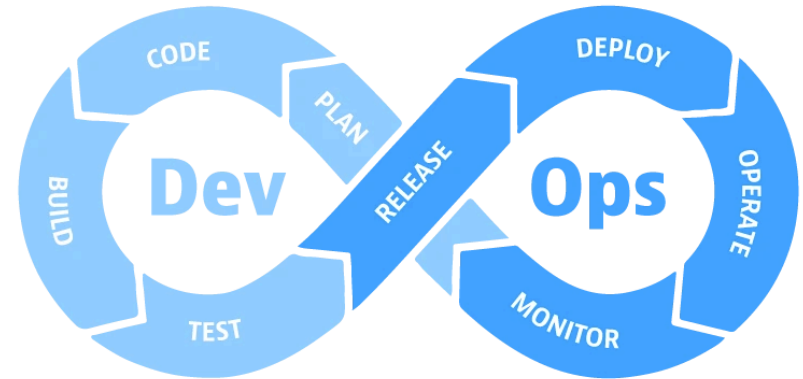
Define el alcance y objetivo de los cambios a realizar.



Conceptos de DevOps

Código: Todo proceso de CI/CD comienza en un repositorio de código al momento de hacer un commit en un branch.

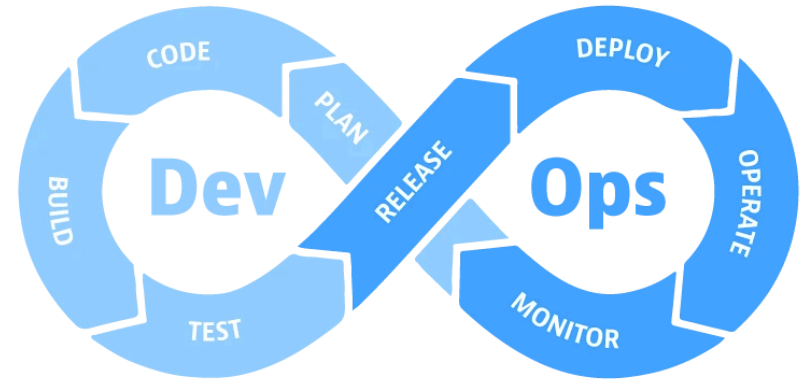
La incorporación de un cambio puede disparar un webhook en la herramienta de CI/CD para iniciar un proceso de Build.



Conceptos de DevOps

Build. Es el proceso de construcción que transforma el código en el elemento que luego será usado para desplegar en producción la aplicación.

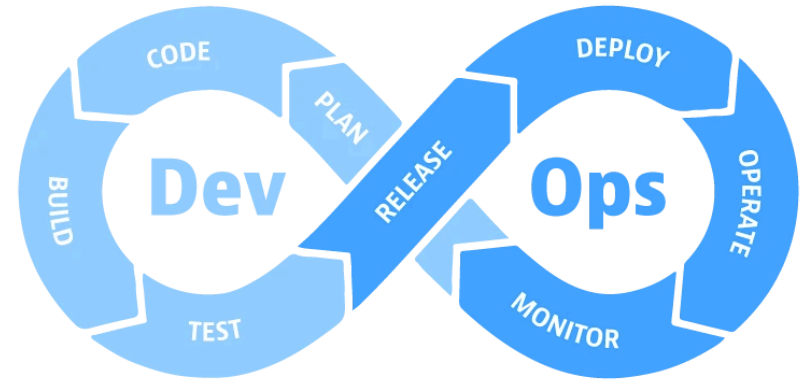
En los ejercicios vistos, las imágenes de contenedores que se crearon son un ejemplo de artefactos generados en esta etapa.



Conceptos de DevOps

Testeo continuo. Involucra un conjunto de test que se pueden hacer sobre los artefactos para verificar que funcionan correctamente y de acuerdo a las especificaciones.

Existen muchos tipos de test y vamos a verlos en detalle en otra clase.



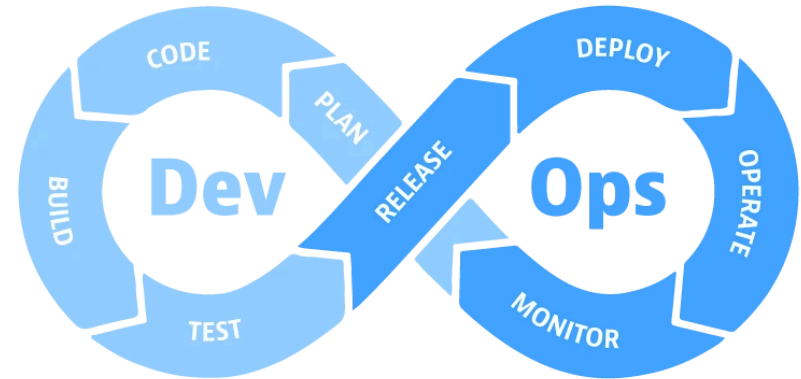
Conceptos de DevOps

Release. Representa una versión de los artefactos habilitada para ser enviada a producción.

Vimos en las imágenes de contenedores que se podía etiquetar para identificar entre otras cosas el release al que pertenecen.

Delivery, es parte del release y corresponde con disponibilizar en producción los artefactos.

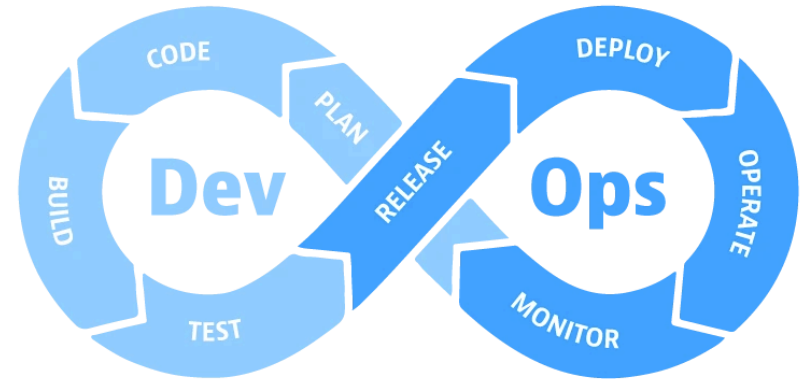
Equivalente a subirlo a la registry.



Conceptos de DevOps

Deploy. Corresponde al proceso de poner en producción un nuevo release.

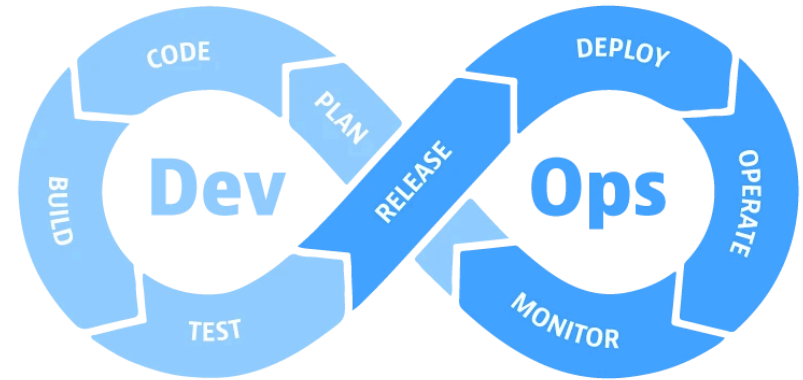
Depende de la arquitectura y plataforma la forma de hacer el deployment.



Conceptos de DevOps

Operate. Corresponde al paso de asegurar la operación de la plataforma.

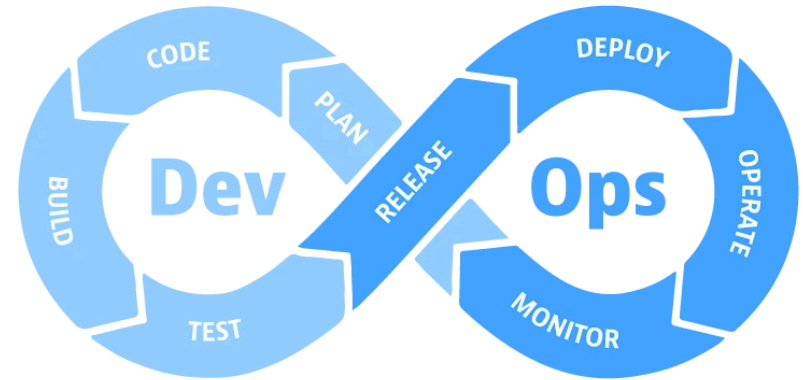
Operación es tener la capacidad de controlar el uso de los recursos dentro de la plataforma y ajustar los parámetros operativos para maximizar su capacidad minimizando el costo.



Conceptos de DevOps

Monitor. Observabilidad de la plataforma.

Capacidad de registrar métricas de operación y eficiencia en la plataforma para hacer una evaluación continua de la calidad del servicio, identificar problemas operativos y de seguridad.



Leftover Principle

The Leftover Principle, developed by Erik Hollnagel, CSE Lab, refers to a trade-off that occurs during mechanization or automation, when the human operator is left with scattered tasks that were not feasible or economic to mechanize or automate.

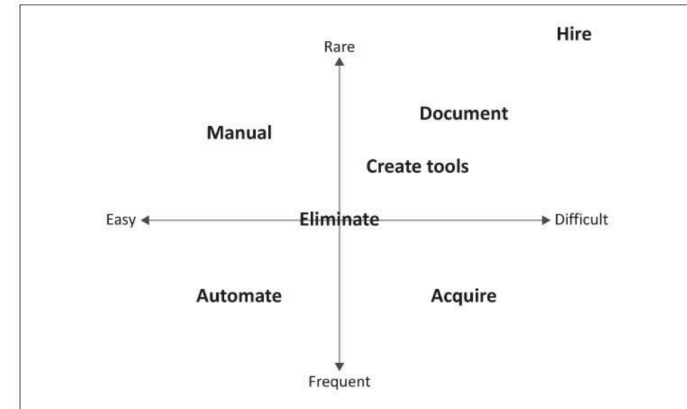
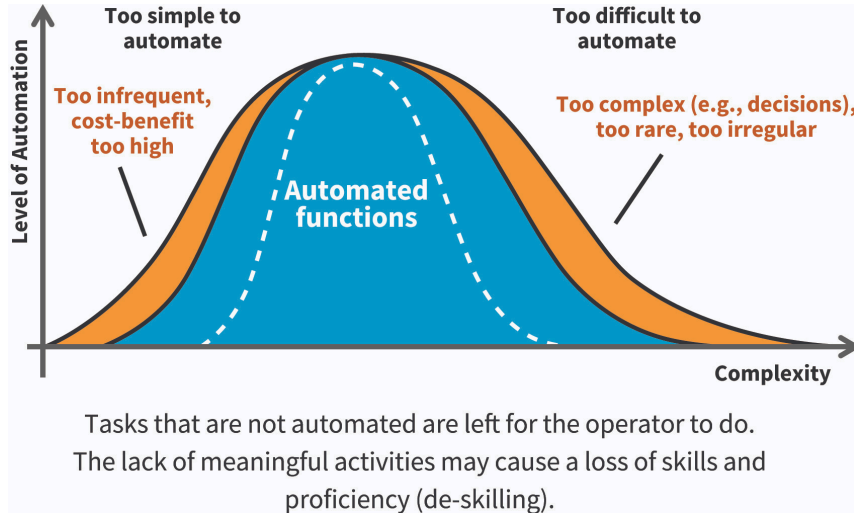
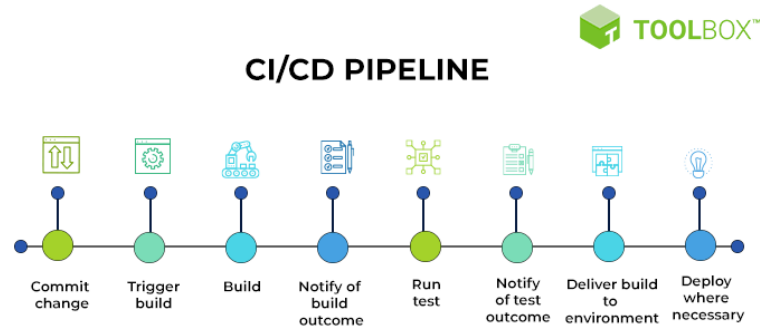


Figure 12.1: Tasks can be classified by effort and frequency, which determines the next step in how to optimize them.

Herramientas de CI/CD

¿Por qué un proceso de CI/CD?

Un proceso de integración y despliegue continuo es una metodología para poner en producción en forma automática una nueva versión de software cada vez que se detecta un cambio en el código.



CI - Continuous Integration

- CI es la práctica de ejecutar el stage de build con cada commit y PR resuelto de una manera automatizada.
- Se basa en usar ramas de feature con vida corta, que contribuyen código a la rama principal.
- El término continuo refiere al hecho de que se testea cada cambio.
- La ventaja de dispara el proceso de build y testing automáticamente es que nadie tiene que estar atento a iniciar el proceso.
- Los errores y el código defectuoso se revelan en corto tiempo.

CD - Continuous Delivery

- CD incorpora todos las prácticas de CI, y además agrega:
 - Test de integración.
 - Tests de performance.
 - UAT
- En un ambiente con CD cada código que pasa por este proceso se considera listo para ser “entregado” es decir desplegado a otros ambientes.
- Pero no incluye el despliegue.

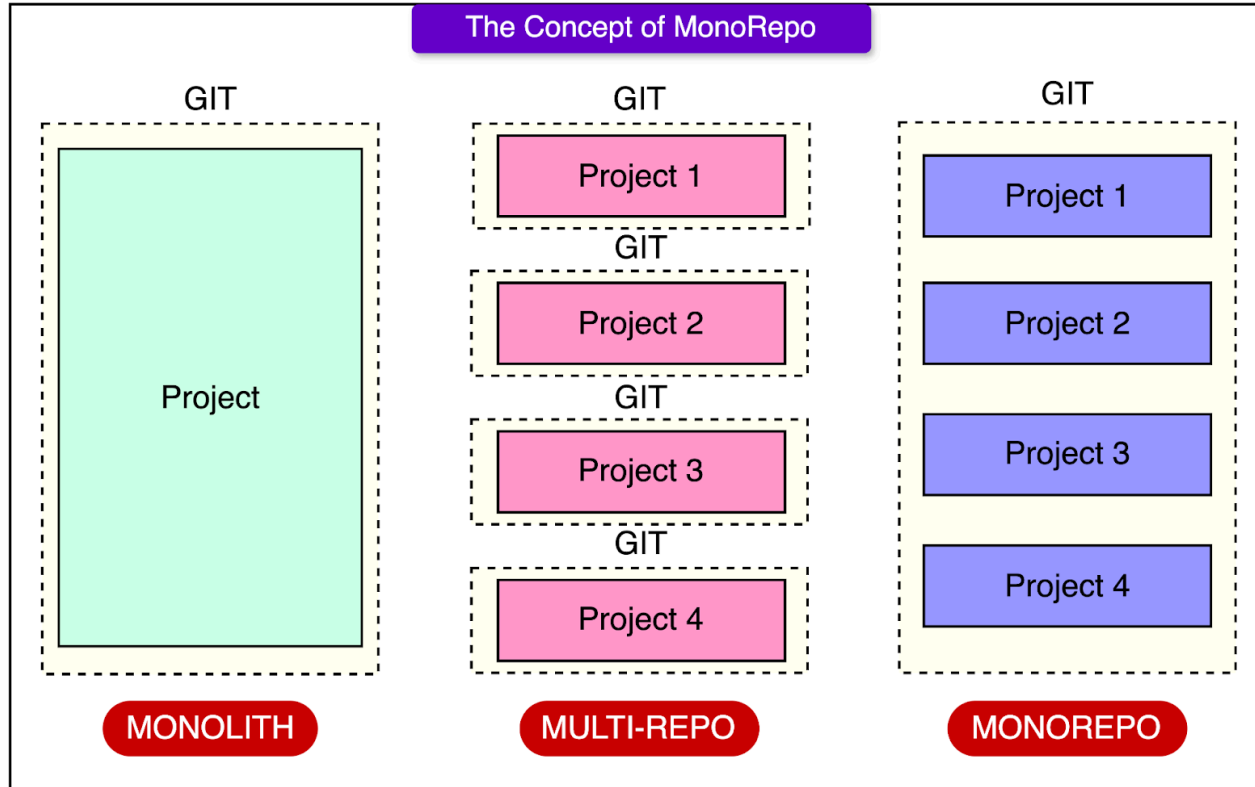
CD - Continuous Deployment

- CD implica que cada release que pasa los tests es desplegado a producción de forma automática.
- Es la meta a alcanzar a menos que la organización esté restringida por regulaciones u otros factores.
- Incluye todo CI/CD más cualquier otra prueba, aprobación y proceso de despliegue para poner el código en producción.
- CD es la decisión de negocio de automatizar la aprobación y el despliegue de releases

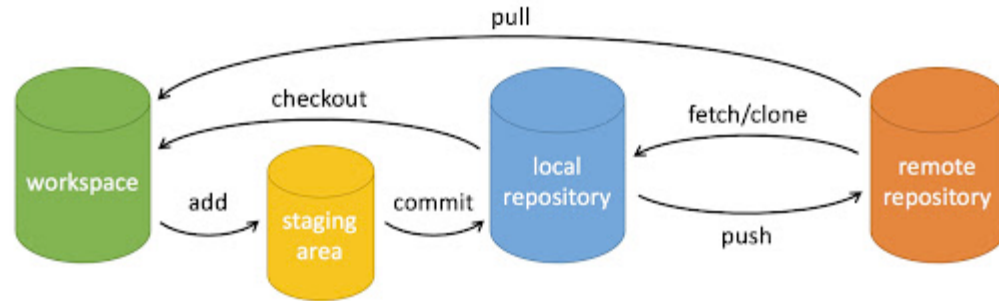
Repositorio de código

- Como su nombre lo indica, un repositorio de código es un lugar en donde se guarda el código.
- No sólo se almacena la última versión, sino que toda la historia de cambio que ha tenido ese código en el tiempo.
- Además, se almacenan las distintas versiones del código, inclusive aquellas que no llegaron a ser implementadas.
- Permite que varias personas trabajen sobre el mismo código y resuelve el problema de juntar distintos cambios sobre el mismo archivo.
- VCS o Version Control Systems, es el nombre técnico con el se identifican los repositorios.

MonoRepo vs Multi-Repo



Uso del repositorio



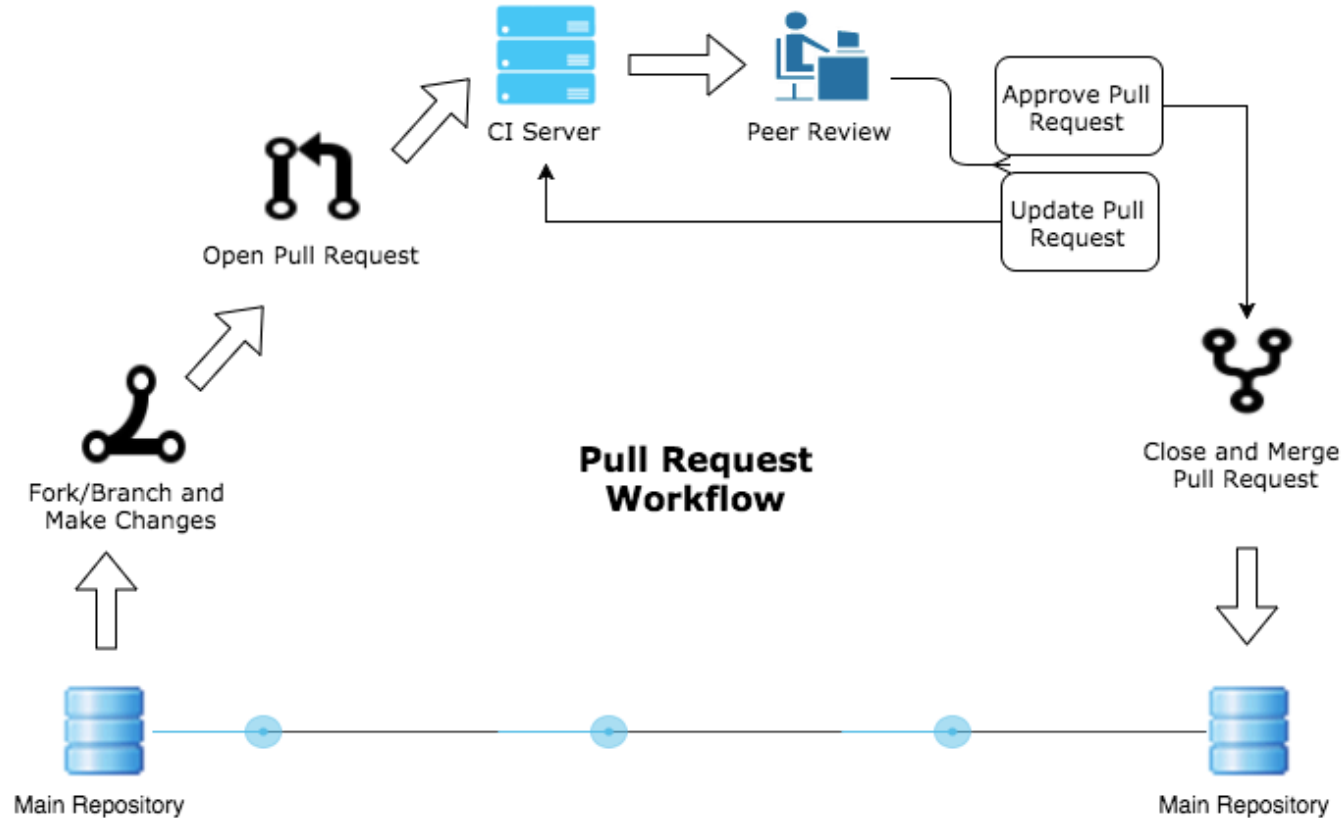
Branches



Pull Request

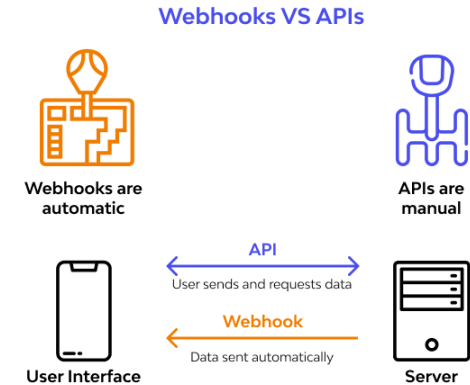
- Un pull request es una discusión que se abre para decidir la incorporación de un cambio en un branch principal.
- En un ambiente de desarrollo colaborativo, más de un desarrollador evalúa el impacto que puede tener esa incorporación al código, hace sugerencias de mejora, aprueba o rechaza el cambio en el código.
- Una vez aprobado, entonces el cambio se le aplica un merge, y pasa a ser parte del branch.
- Los repositorios modernos pueden incorporar controles automáticos como parte del proceso de aprobación de pull request.

Pull Request Workflow



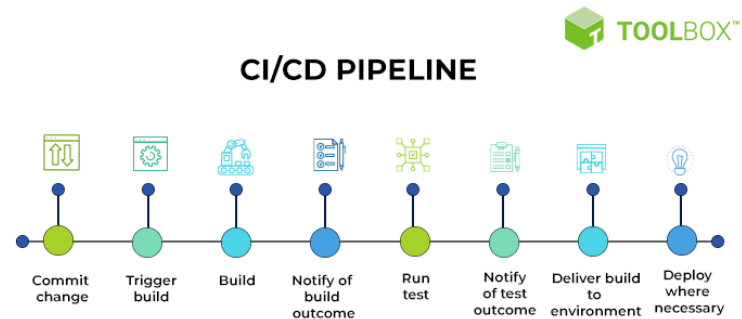
Repository Webhooks

- Los servicios de SCM (source code management) como GitHub, GitLab, etc ofrecen muchas extensiones y capacidades de integración con otros servicios de seguridad, de CI/CD entre otras funcionalidades.
- La forma de integración los SCM y el resto de los sistemas típicamente se realiza por medio de Webhook y la API de este servicio.



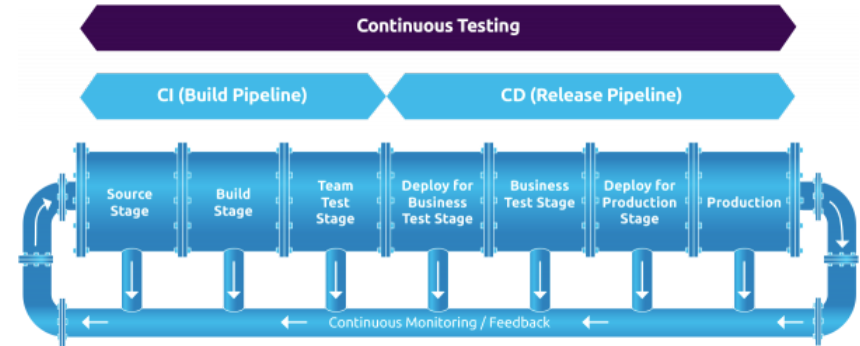
SCM y el CI/CD

- En todos los estados de los pull request los webhooks pueden integrarse a herramientas de CI/CD para disparar procesos de control.
- Uno de los controles típicos corresponde a verificar si ese cambio concluye exitosamente el proceso de build y/o testing.
- El resultado del proceso se refleja en el mismo pull request y ayuda a dar feedback para la decisión de integrar el cambio.



Pipeline

- Un pipeline es una secuencia de pasos que va llevando adelante el proceso de CI/CD y antes cualquier falla, poder volver al inicio y corregirla.
- Los conceptos de Fail Fast, Fail First son críticos para corregir los problemas encontrados.



Ejemplos de Pipeline

```
node {
    // Get Artifactory server instance, defined in the Ar
    def server = Artifactory.server "SERVER_ID"
    // Create an Artifactory Gradle instance.
    def rtGradle = Artifactory.newGradleBuild()
    def buildInfo

    stage('Clone sources') {
        git url: 'https://github.com/jfrogdev/project-exa
    }

    stage('Artifactory configuration') {
        // Tool name from Jenkins configuration
        rtGradle.tool = "Gradle-2.4"
        // Set Artifactory repositories for dependencies
        rtGradle.deployer repo:'ext-release-local', server: s
        rtGradle.resolver repo:'remote-repos', server: s
    }

    stage('Gradle build') {
        buildInfo = rtGradle.run rootDir: "gradle-example
    }

    stage('Publish build info') {
        server.publishBuildInfo buildInfo
    }
}
```

```
1 build-job:
2   stage: build
3   script:
4     - echo "Hello, $GITLAB_USER_LOGIN!"
5
6 test-job1:
7   stage: test
8   script:
9     - echo "This job tests something"
10
11 test-job2:
12   stage: test
13   script:
14     - echo "This job tests something, but take
15     - echo "After the echo commands complete,
16     - echo "which simulates a test that runs 2
17     - sleep 20
18
19 deploy-prod:
20   stage: deploy
21   script:
22     - echo "This job deploys something from th
23
```

Stages & Steps

- **Stage:** Etapa en el pipeline. Una etapa tiene una función propia y suele componerse de uno o más jobs o steps.
- **Job (Steps):** Un paso en el proceso de ejecución de una stage. Define lo que hay que hacer-
- **Runners (Agentes):** Ambiente en donde se ejecutan los steps de un stage. Puede ser una máquina local, en la nube o remota (SaaS) dentro del proveedor de CI/CD.

Ejemplo GitLab

- Actualmente existen cientos de soluciones para CI/CD.
- Más allá de que todas comparten principios similares, cada uno tiene sus términos y características propias.
- Vamos a tomar de referencia GitLab para este curso.
- GitLab funciona ejecutando en un contenedor cada uno de los pasos. Los pasos se ejecutan en ambientes aislados. Para intercambiar artefactos entre los steps, se usa el cache o los artefactos.
- Cada paso puede exportar uno o más artefactos (archivos).
- Cada pipeline puede compartir un cache, para evitar crear contenido reutilizable, como librerías de código o imágenes de contenedores.

Etapas típicas de CI/CD

- Preparation
- Build
- Unit Test
- Deliver
- Integration Test
- Deploy

Pipeline: Preparation

- Involucra la preparación del ambiente de trabajo.
 - Seteo de las versiones correctas de software
 - Búsqueda de credenciales y secretos necesarios para las siguientes etapas
 - Recolección de otros artefactos necesarios (repositorios)
- A fin de poder asegurar los mismos resultados bajo las mismas condiciones, la preparación es crítica para crear el mismo ambiente para cada ejecución del pipeline.
- El uso de Configuración como Código para el ambiente de pipeline permite reproducir siempre el mismo estado inicial, para asegurar el mismo estado final.

Pipeline: Build

- Dependiendo del lenguaje usado, serán los pasos realizados en la etapa de construcción.
 - Compilación
 - Resolución de dependencias
 - Creación de paquetes (módulo y librerías)
- Normalmente esta etapa es la que genera los artefactos usados en el resto de las etapas.
- Podría generar artefactos específicos para testing y artefactos específicos (debugging) para producción.
- Todo artefacto podría ser un release candidate, y por lo tanto debe preservarse hasta que sea rechazado.

Pipeline: Unit Test

- Testeo de los artefactos en su forma más simple. Debe ser un proceso incremental en la cantidad y calidad de las pruebas.
- Debe poder confirmar que nuevas versiones no rompen viejos comportamientos esperados.
- A nivel de microservicios, un unit test podría estar focalizado en el microservicio en sí. Cada endpoint de la API
 - ¿Cómo responde la API?,
 - ¿Realiza los cambios esperados en el DB?.
 - ¿Queda corriendo el servicio tras su ejecución?
- La etapa de unit test podría requerir de un ambiente de preparación para el ambiente de testing.

Pipeline: Deliver

- Deliver es la disponibilización de los artefactos construidos en los distintos ambientes.
- El artefacto en si no se modifica, pero puede llegar a ser necesario modificar la configuración que define qué artefacto debe ser usado, por ejemplo, un docker-compose.yaml.
- Ejemplos de delivery:
 - Subir una imagen a una registry.
 - Publicar paquetes de librerías y ejecutables en un disco compartido
 - Publicar paquetes en repositorio de paquetes tipo npm o PyPy

Pipeline: Integration Test

- Testeo de integración debe simular el ambiente de producción y componer un ambiente con la nueva versión de todos los artefactos.
- Debe poder simular operaciones complejas en el ambiente, para verificar que se ejecutan correctamente de punta a punta.
- Son test muy complejos, y su diseño lleva a veces más tiempo que el desarrollo de la aplicación en sí.
- Normalmente se ejecutan de a partes, probando integraciones parciales, y no para todos los releases, sino solamente en aquellos que introducen muchos cambios.

Pipeline: Deploy

- Deploy es la acción de poner en producción un cambio.
- Dependiendo del tipo de software y su arquitectura existen distintas estrategias para hacer un deploy.
- Algunas tecnologías, como Kubernetes incluyen procesos que simplifican el proceso de deploy.
- Un deploy podría incluir algo tan simple como cambiar una imagen en una registry, de modo que la siguiente vez que arranque ese contenedor arranque con la imagen nueva, o incluir modificación de una base de datos, el reinicio de una máquina e incluso una ventana sin servicio en la aplicación.

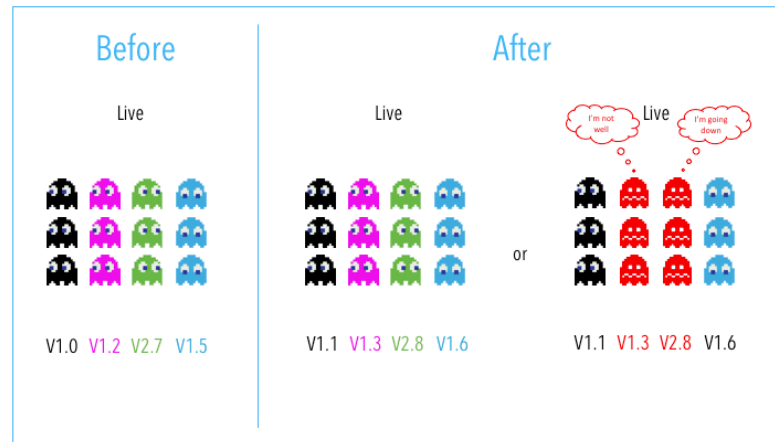
Deployment: Basic

- Es la estrategia más simple.
- Todos los nodos se bajan, se actualizan y se vuelven a levantar.
- Desde el diseño es la estrategia más simple, pero la más riesgosa. No hay servicio durante el proceso de despliegue, y si fuera necesario hacer un roll-back, se dan vuelta atrás todos los cambios.



Deployment: Multi Service Deployment

- Despliegue de múltiples servicios al mismo tiempo. Cada servicio con su nueva versión.
- A veces es necesaria si hay mucho acoplamiento entre servicios.
- Hace las pruebas de integración muy complejas y aumenta las posibilidades de fallos.



Deployment: Rolling

- En un conjunto de nodo, se van actualizando en forma progresiva, hasta reemplazar la versión de todos los nodos.
- Es un proceso automático y no hay una evaluación del éxito del despliegue hasta finalizar el mismo.
- Es fácil de implementar, y fácil su roll-back.
- Debe asegurar retrocompatibilidad en el software, de modo que distintas versiones puedan convivir.

State 0



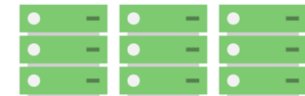
State 1



State 2

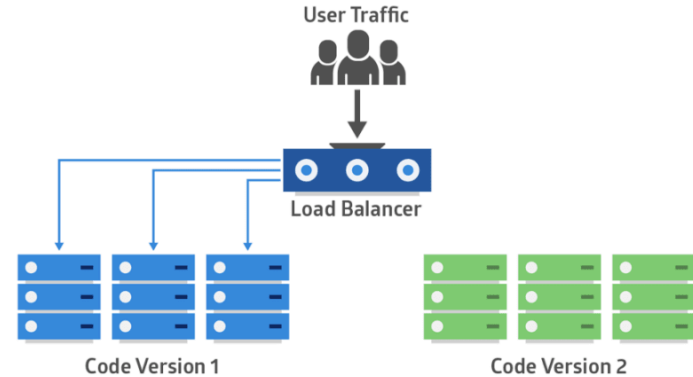


Final State



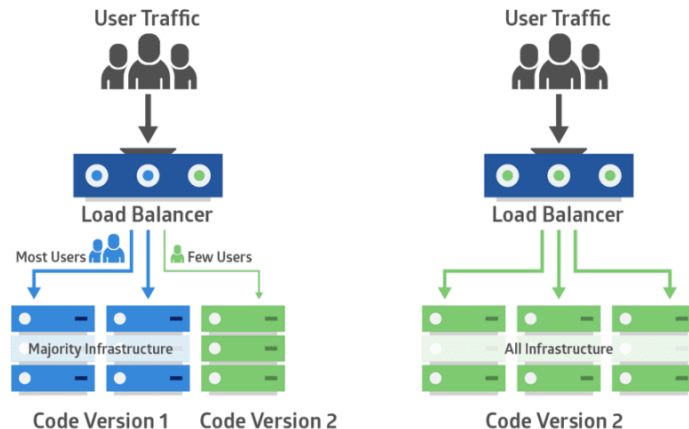
Deployment: Blue / Green

- Es la estrategia más simple.
- Se basa en la idea de armar un nuevo ambiente paralelo al que está operativo, y por medio de algún distribuidor de tráfico controlar a cual de los dos ambientes se distribuye el tráfico.
- Es uno de los más usados en ambiente de contenedores simples.



Deployment: Canary

- Se basa en la idea de distribuir a las nuevas versiones a un grupo de usuarios, mientras que el resto sigue operando con la versión anterior.
- Progresivamente, cuando se va verificando que no hay problemas, se van actualizando nuevos nodo y distribuyendo más usuarios hacia la nueva versión hasta completar el despliegue total.
- Asegura un deployment progresivo, de bajo impacto ante fallas y rápido roll-back

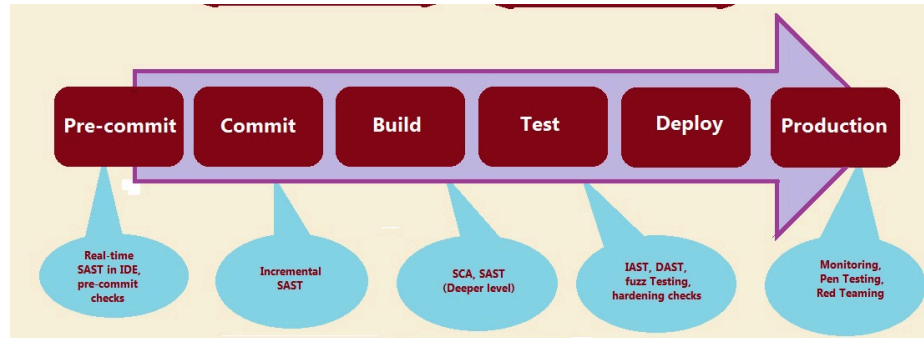


Estrategia de despliegue

- La estrategia de despliegue correcta dependerá del riesgo que estemos dispuestos a correr y el impacto en el negocio que podría tener ese error.
- La estrategia de despliegue del software que maneja el firmware de un respirador artificial, no debe ser la misma que la que usa el Call of Duty.
- Cada estrategia tiene sus costos, de modo que debe ser parte de los criterios a ser considerados a la hora de diseñar la arquitectura de la solución, y la estrategia de CI/CD.

Seguridad como parte del CI/CD

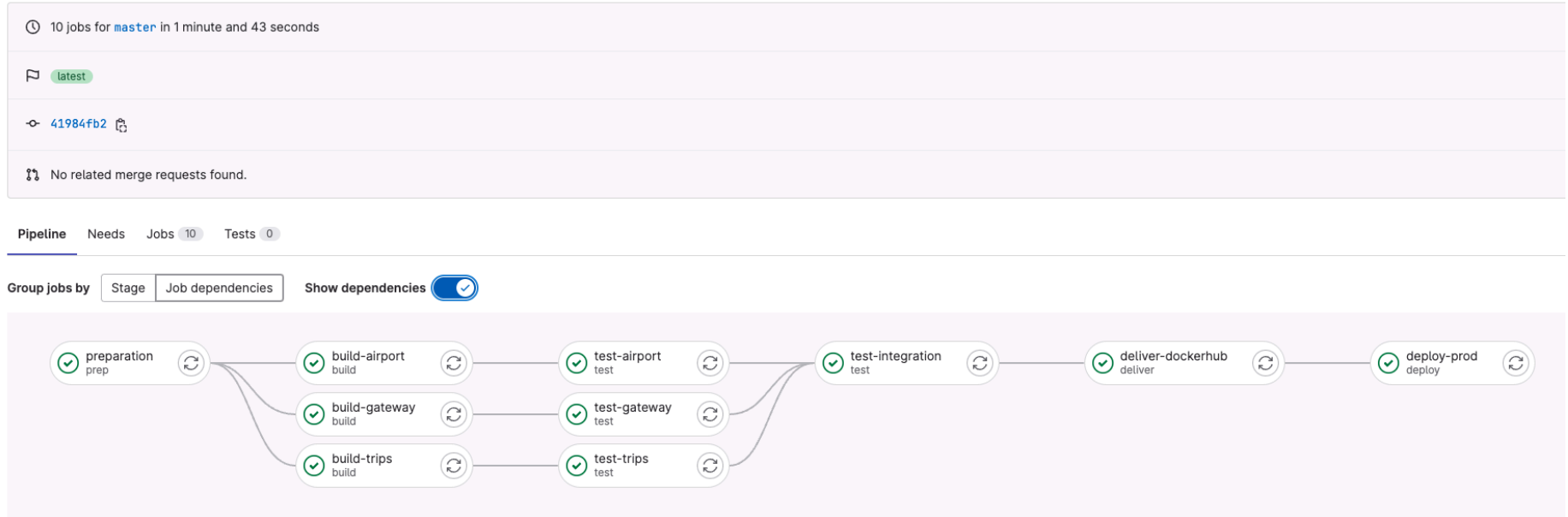
- La seguridad del software hoy en día es tan crítica como su disponibilidad y debe ser parte de las evaluaciones que se realizan sobre el mismo.
- Estos controles pueden incorporarse dentro del pipeline y bloquear un despliegue de módulos, imágenes, configuraciones o código inseguro.



Shift-Left

- El concepto de Shift-Left significa devolver al desarrollador cuanto antes un error detectado.
- El desarrollador hace un commit o un merge a main, disparando un pipeline.
- El pipeline realiza todos los test funcionales , de performance y de seguridad, y ante cualquier falla, inmediatamente se le avisa al desarrollador del error.
- La cultura del shift left en los procesos de desarrollo le da la responsabilidad al desarrollador de solucionar los problemas apenas surgen, y no esperar a que el problema llegue a producción.
- Los procesos shift-left permiten reducir los costos de operación, al focalizarse en los problemas antes de que el software llegue a producción.

Ejemplo GitLab



Fin!!